



Universidad Tecnológica del Uruguay (UTEC)

Proyecto Integrador — Brazo robótico didáctico

Evidencias de proyecto

Integrantes: Mateo Lecuna, Hector Pereira, Priscila Rossi

Docente/s: Marcelo Diaz, Christian Campero

Fecha: 26 de octubre de 2025

Índice

1. Resumen	2
2. Metodología y cronograma	2
3. Evidencias de diseño y fabricación	2
3.1. Materiales y métodos	2
3.2. Brazo articulado	2
3.3. Controlador espejo	6
4. Electrónica y programación	7
4.1. Electrónica de control	7
4.2. Interfaz de usuario	8
A. Repositorio de evidencias digitales	10
B. Código	10
B.1. Control de Servomotor con Modo Potenciómetro y Comunicación Serial (USART) . . .	10
B.2. Control de Dos Servomotores con Potenciómetros (Control Espejo)	13
B.3. Programa Principal del Brazo Robótico Didáctico	15
B.4. Interfaz de Usuario	19

1. Resumen

2. Metodología y cronograma

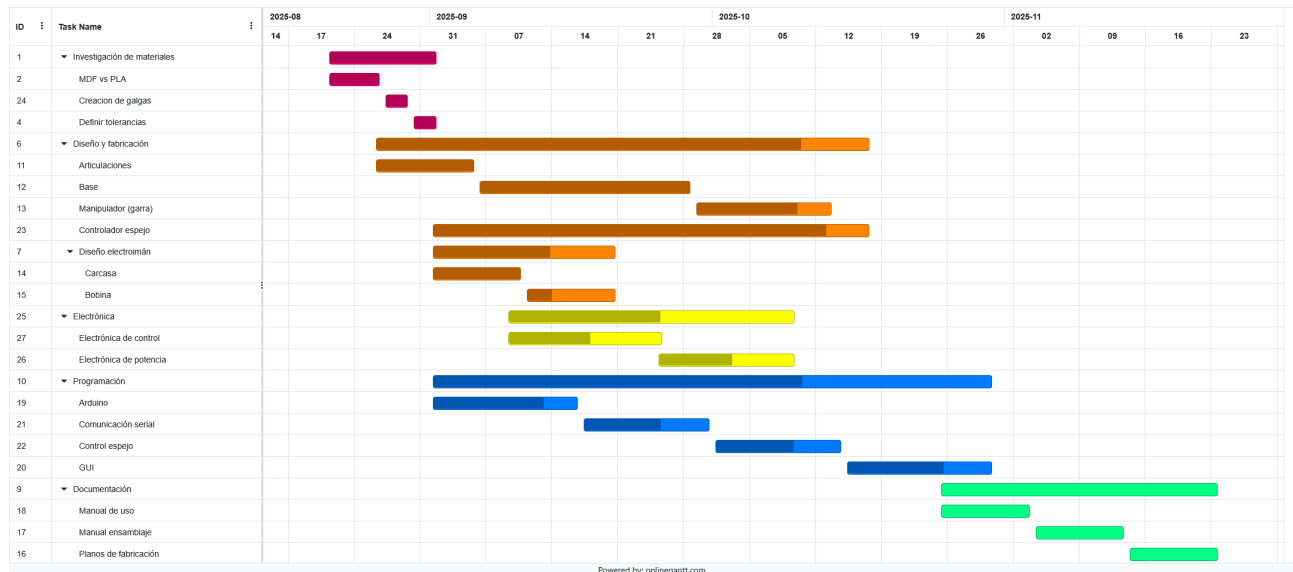


Figura 1: Diagrama de Gantt. Fuente: elaboración propia

3. Evidencias de diseño y fabricación

3.1. Materiales y métodos

Para el desarrollo del brazo robótico se emplearon materiales accesibles y técnicas de fabricación complementarias, priorizando la modularidad y facilidad de ensamblaje. Las primeras etapas se centraron en el uso de MDF cortado por láser para los componentes estructurales, debido a su precisión y rapidez de fabricación, mientras que las piezas que requerían geometrías más complejas o encastrados finos se imprimieron en 3D con filamento PLA.

El diseño se realizó íntegramente en software CAD, permitiendo iterar entre versiones y evaluar virtualmente la cinemática de cada articulación. Posteriormente, se llevaron a cabo pruebas físicas con prototipos a escala real para validar tolerancias, rigidez y facilidad de ensamblaje antes de consolidar el diseño final.

3.2. Brazo articulado

Se realizaron cortes en MDF de los tres primeros segmentos articulados del brazo con el fin de evaluar tolerancias de fabricación, rigidez estructural y dificultad de ensamblaje. Durante esta etapa se verificó el calce de los servomotores y articulaciones, así como la estabilidad del conjunto al aplicar cargas simuladas.

Los resultados mostraron que el espesor del material utilizado era insuficiente para garantizar la rigidez necesaria, especialmente en los puntos de articulación. Además, el tipo de encastrado abierto empleado en esta primera versión dificultaba el armado, ya que las piezas tendían a salirse de posición durante el proceso de montaje.

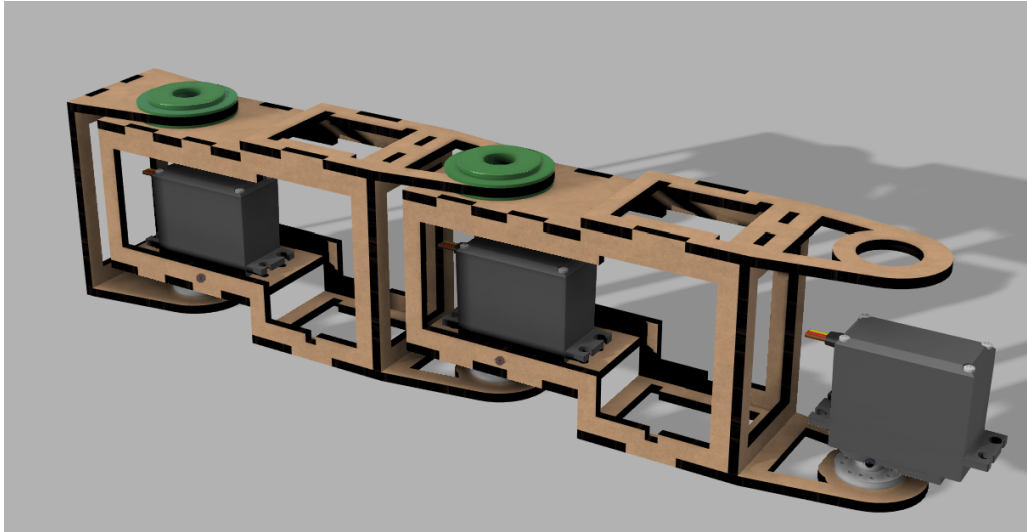


Figura 2: Corte y ensamblaje preliminar de los tres primeros segmentos en MDF para evaluar tolerancias, rigidez y dificultad de ensamblaje. Fuente: elaboración propia.

A partir de estas observaciones, se rediseñaron los componentes con un sistema de encastre envolvente, el cual permite mantener las piezas en posición estable antes del pegado o atornillado. Asimismo, se incorporó una base con patas y una superficie destinada al montaje de los componentes electrónicos, mejorando la distribución del peso y el acceso durante el mantenimiento. Se añadieron también monturas adicionales para la fijación de tornillos y la instalación de medios elásticos —como gomas o resortes— que asisten al servomotor de la articulación inferior en ángulos de apertura críticos.

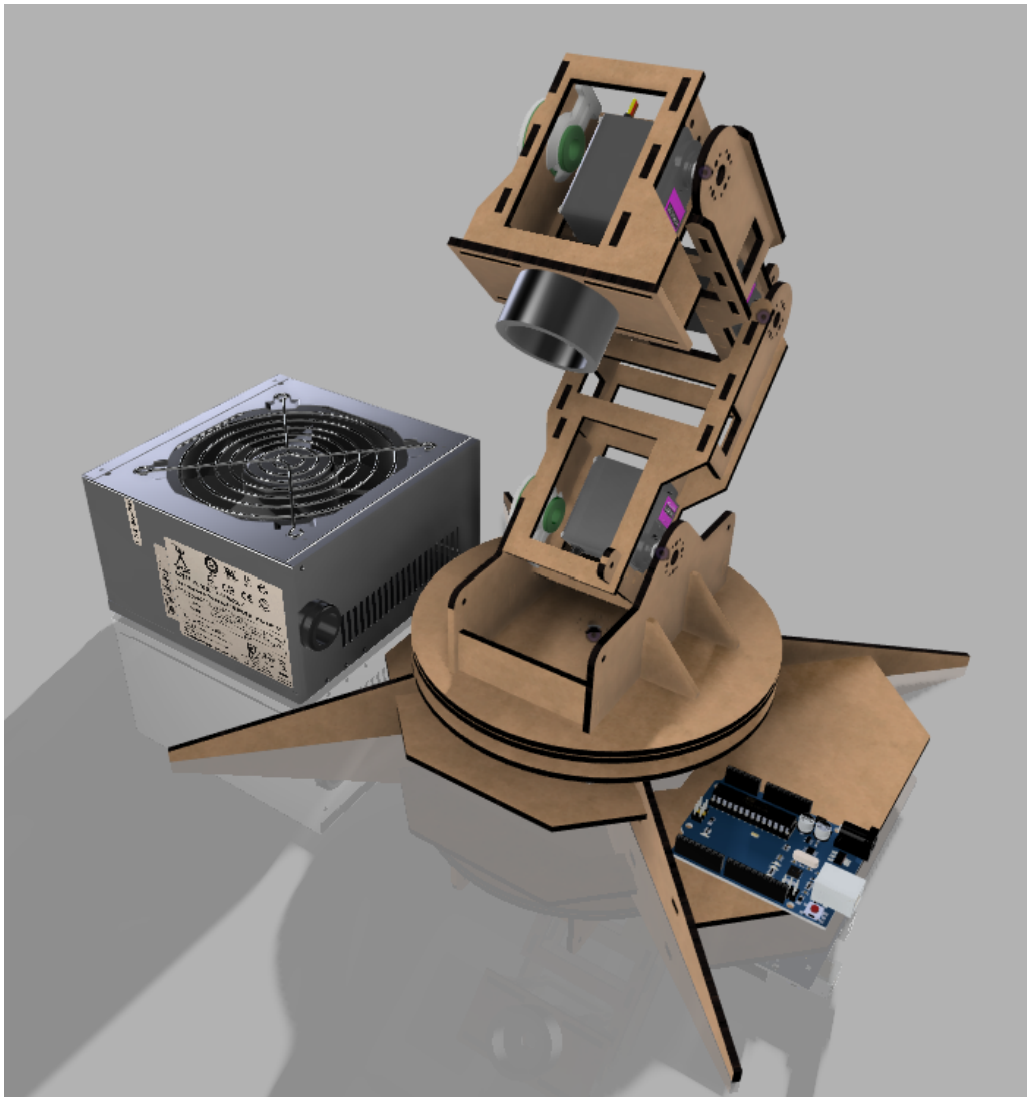


Figura 3: Render del diseño final del brazo robótico, con encastrés envolventes y base estructural reforzada. Fuente: elaboración propia.

El modelo final presentó una notable mejora en rigidez y estabilidad durante el ensamblaje. Las gomas elásticas implementadas resultaron eficaces para reducir el esfuerzo sobre los servomotores, evitando la necesidad de sustituirlos por modelos de mayor torque. Finalmente, se realizó una prueba funcional con el electroimán montado, utilizando una galga reciclada de un diseño anterior para estimar el comportamiento dinámico y la capacidad de carga del brazo.

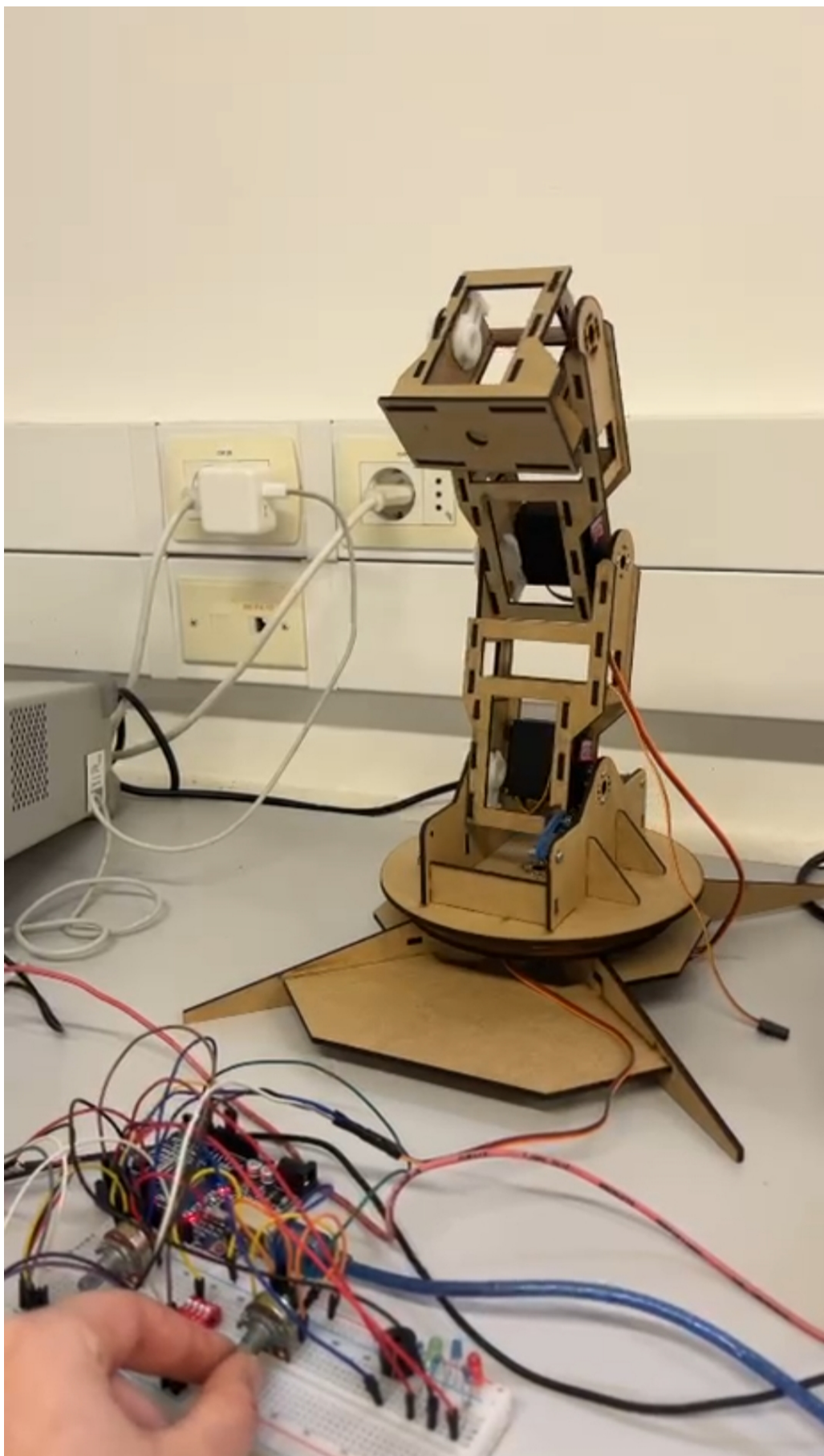


Figura 4: Modelo final ensamblado del brazo articulado con gomas elásticas de asistencia y prueba del electroimán. Fuente: elaboración propia.

3.3. Controlador espejo

El controlador del brazo fue diseñado como una réplica funcional que permite manipular directamente las articulaciones del modelo principal mediante un sistema de control espejo. En su primera versión, se optó por una estructura completamente fabricada en PLA impreso en 3D, lo que ofrecía una alta precisión dimensional pero presentaba limitaciones en rigidez y tiempos de fabricación prolongados.

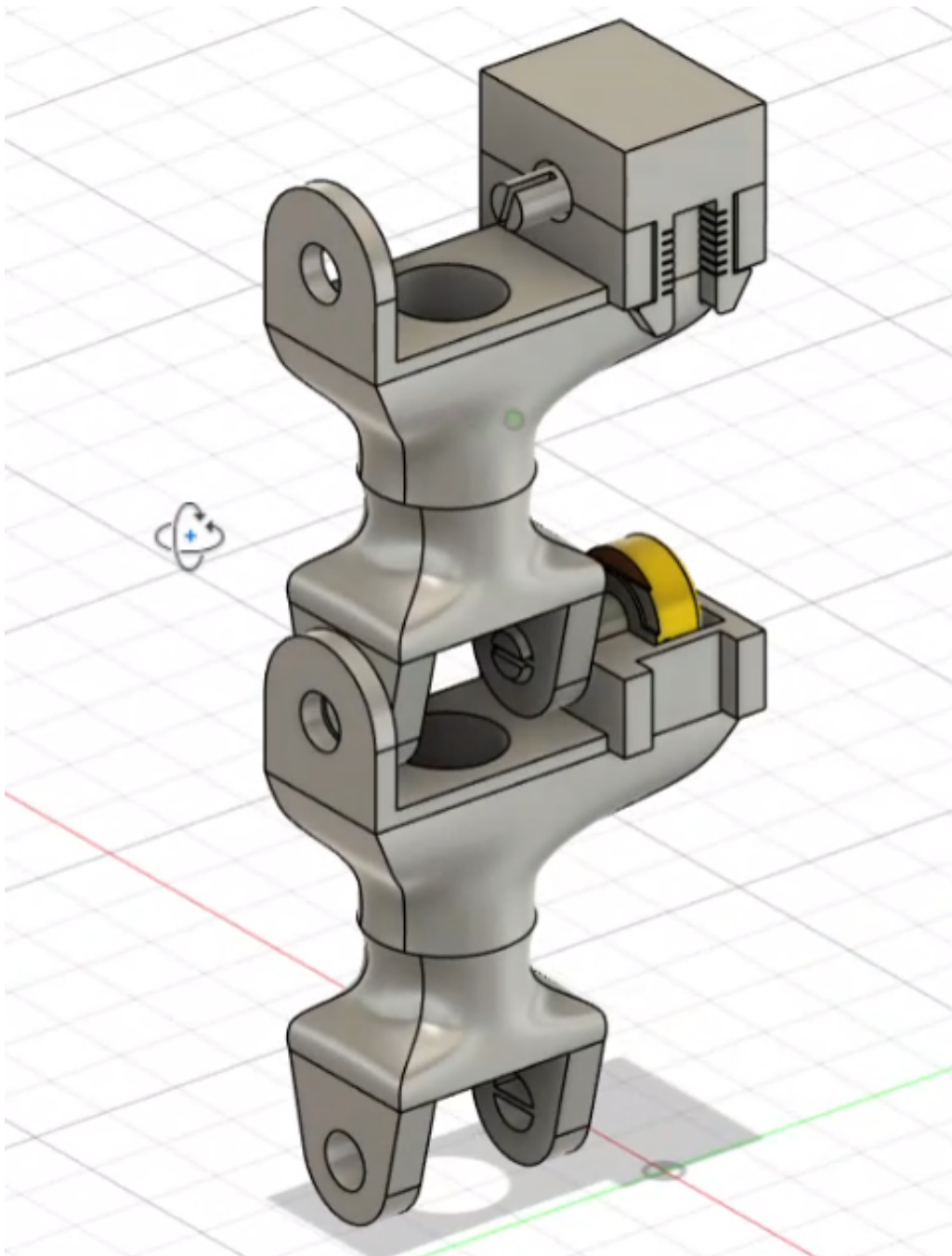


Figura 5: Diseño inicial del controlador del brazo fabricado completamente en PLA. Fuente: elaboración propia.

Frente a estas limitaciones, se implementó un rediseño híbrido que combina MDF y PLA, aprovechando las ventajas de ambos métodos. La estructura principal se construyó en MDF mediante corte láser, garantizando una base robusta y de rápida producción, mientras que los encastres, acoples y carcasas con geometrías más complejas se imprimieron en PLA.

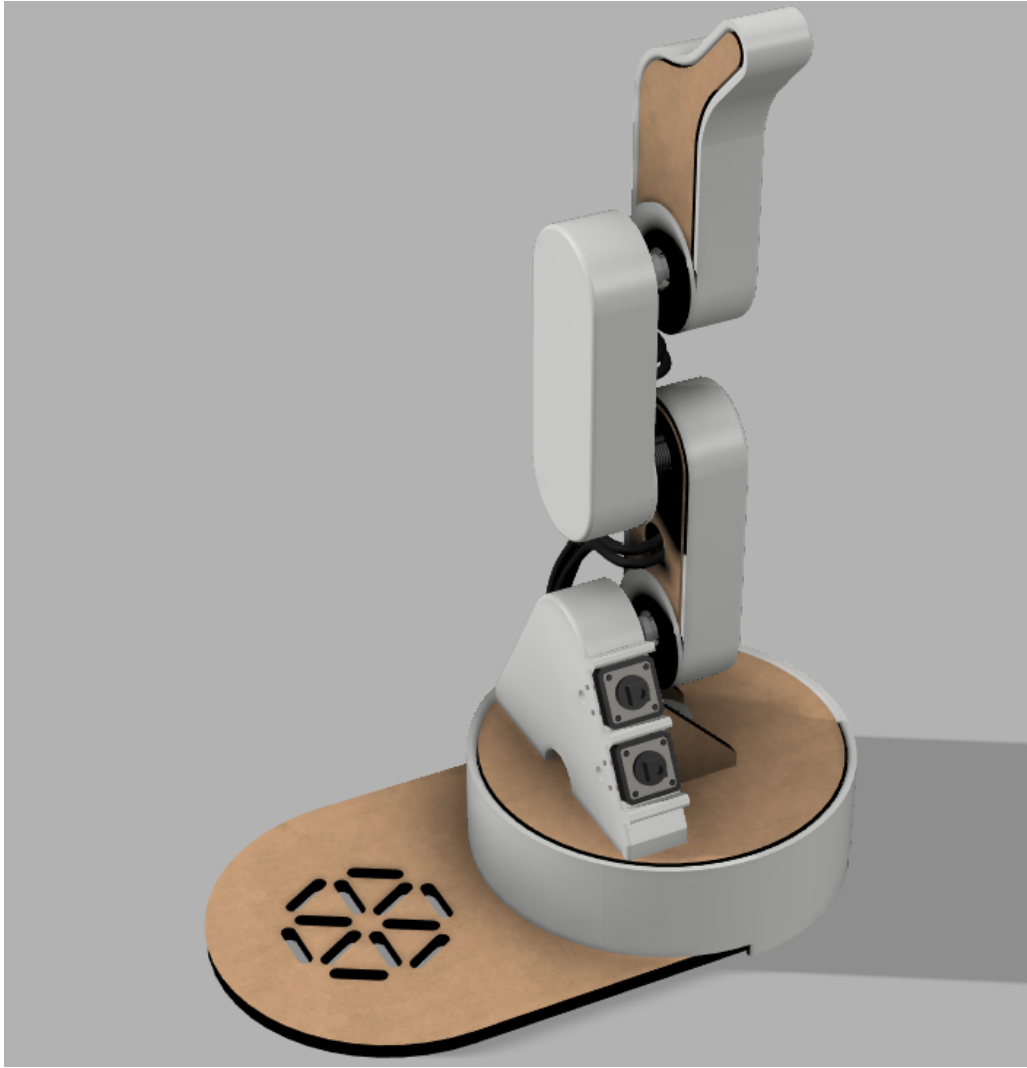


Figura 6: Nuevo diseño del controlador híbrido MDF-PLA, combinando corte láser e impresión 3D para optimizar rigidez y precisión. Fuente: elaboración propia.

El diseño final logra una integración eficiente entre las piezas de madera y plástico, garantizando la precisión mecánica necesaria para reproducir los movimientos del brazo robótico. Además, el enfoque modular del diseño facilita futuras mejoras, como la incorporación de sensores o la sustitución de componentes específicos sin necesidad de rehacer toda la estructura.

4. Electrónica y programación

4.1. Electrónica de control

El sistema de control del brazo robótico está basado en un microcontrolador **ATmega328P** montado sobre una placa **Arduino Uno**, el cual actúa como unidad principal de procesamiento y coordinación de señales. En el circuito se implementa una arquitectura de tipo *control espejo*, donde un conjunto de potenciómetros permite reproducir manualmente los movimientos que el brazo ejecuta mediante servomotores.

Cada potenciómetro funciona como un divisor de tensión conectado entre 5.00 V y tierra, entregando en su terminal central una señal analógica proporcional al ángulo de giro. Estas señales son adquiridas por las entradas analógicas del microcontrolador (A0-A3), digitalizadas mediante su convertidor analógico-digital (ADC) interno y posteriormente traducidas a señales de modulación por ancho de pulso (PWM) generadas en los pines digitales (D6-B1-B2). Cada una de estas salidas PWM gobierna directamente un servomotor, logrando así que la posición de los ejes del brazo robótico imite en tiempo real el movimiento realizado sobre los potenciómetros del controlador.

La alimentación del sistema se distribuye a través de una línea común de 5.00 V y una referencia de tierra compartida. En esta configuración, los servomotores, potenciómetros y el microcontrolador comparten la misma fuente de energía, garantizando la coherencia de referencia eléctrica. No obstante, para versiones finales o de mayor carga mecánica, se considera la posibilidad de incorporar una fuente externa de corriente continua dedicada exclusivamente a los servomotores, a fin de evitar caídas de tensión durante el funcionamiento.

Finalmente, el esquema incluye tres indicadores luminosos —**LED rojo, verde y azul**— que cumplen funciones de depuración y señalización visual. Estos se conectan a través de resistencias limitadoras de corriente y permiten indicar estados del sistema como encendido, operación correcta o condiciones de error.

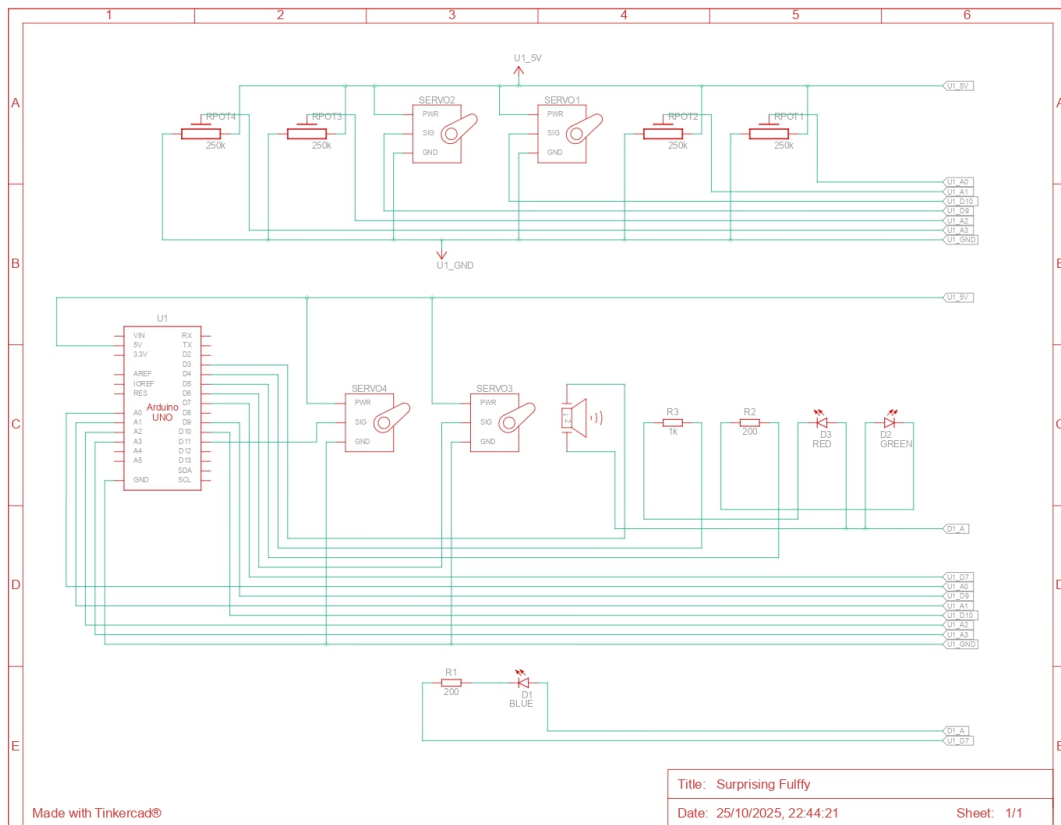


Figura 7: Diagrama eléctrico del sistema de control del brazo robótico, con conexiones entre potenciómetros, servomotores, LEDs y microcontrolador Arduino Uno. Fuente: elaboración propia.

4.2. Interfaz de usuario

La interfaz de usuario fue desarrollada en **Python** utilizando la biblioteca **Tkinter**, con el propósito de ofrecer un medio visual e intuitivo para interactuar con el sistema. Desde esta aplicación, el usuario puede seleccionar entre dos modos de funcionamiento: **automático** y **manual**, ambos comunicados con el microcontrolador a través del protocolo **USART**.

En el **modo automático**, el programa ejecuta secuencias predefinidas de movimiento controlando los ángulos de cada servomotor directamente desde el entorno Python. Los valores de posición se transmiten mediante el puerto serial al microcontrolador, que interpreta los comandos y ajusta los servos en consecuencia. Este modo resulta útil para la ejecución repetitiva de trayectorias o demostraciones programadas sin intervención del usuario.

Por otro lado, el **modo manual** permite al operador controlar en tiempo real la posición de cada

articulación a través de los potenciómetros del controlador espejo. En este caso, la interfaz cumple el rol de monitor serial, mostrando los valores de ángulo recibidos desde el microcontrolador y permitiendo el cambio de estado o calibración mediante comandos específicos.

El seteo de ángulos, el cambio entre modos de operación y la comunicación bidireccional entre Python y el microcontrolador se gestionan completamente a través del puerto serial, utilizando mensajes de texto estructurados y delimitadores que aseguran la correcta interpretación de los datos en ambos sentidos. Esta implementación facilita la depuración del sistema y ofrece una capa adicional de control y supervisión sobre el hardware embebido.

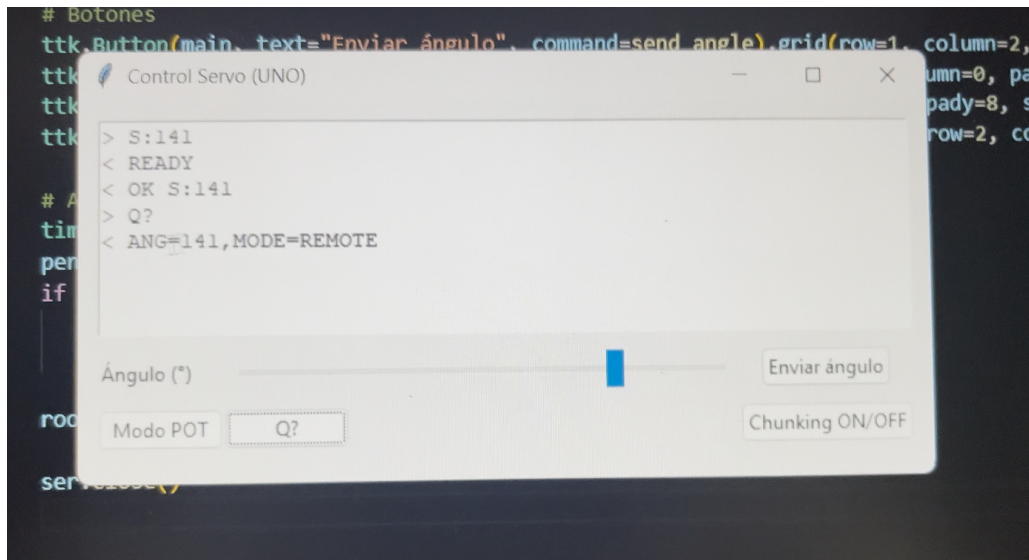


Figura 8: Interfaz de usuario desarrollada en Python utilizando la biblioteca Tkinter, con opciones de control automático y manual comunicadas por USART. Fuente: elaboración propia.

A. Repositorio de evidencias digitales

Con el fin de centralizar y facilitar el acceso a todos los materiales complementarios del proyecto, se ha dispuesto un repositorio en línea que contiene los archivos digitales correspondientes a las evidencias presentadas en este informe. Dicho repositorio incluye fotografías del proceso de fabricación, renders de los modelos 3D, archivos CAD, esquemas eléctricos, simulaciones, programas en C y Python, así como los recursos multimedia asociados a las pruebas funcionales.

El acceso al repositorio puede realizarse mediante el siguiente enlace:

[Repositorio en Google Drive — Evidencias del Proyecto](#)

B. Código

B.1. Control de Servomotor con Modo Potenciómetro y Comunicación Serial (USART)

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <stdio.h>

/* ===== Servo (D9/0C1A) ===== */
#define SERVO_MIN_US 500
#define SERVO_MAX_US 2500
#define SERVO_SLEW_STEP 2

static inline void servo_init(void){
    DDRB |= (1<<PB1);
    TCCR1A = (1<<COM1A1) | (1<<WGM11);
    TCCR1B = (1<<WGM13) | (1<<WGM12) | (1<<CS11); // presc=8
    ICR1 = 39999; // 20 ms
}

static inline uint16_t us_to_ticks(uint16_t us){ return (uint16_t)(us*2); }
static inline void servoA_write_us(uint16_t us){
    if(us<SERVO_MIN_US) us=SERVO_MIN_US;
    if(us>SERVO_MAX_US) us=SERVO_MAX_US;
    OCR1A = us_to_ticks(us);
}

static inline uint16_t deg_to_us(uint8_t deg){
    if(deg>180) deg=180;
    return (uint16_t)(SERVO_MIN_US + ((uint32_t)(SERVO_MAX_US-SERVO_MIN_US)*deg)/180);
}

/* ===== ADC (A0) ===== */
#define ADC_SAMPLES 10
#define ADC_DEADBAND 2
static inline void adc_init(void){
    ADMUX = (1<<REFS0);
    ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1); // /64
}

static inline uint16_t adc_read(uint8_t ch){
    ADMUX = (ADMUX & 0xF0) | (ch & 0x0F);
    ADCSRA |= (1<<ADSC);
    while(ADCSRA & (1<<ADSC));
}
```

```

        return ADC;
    }
    static inline uint16_t adc_read_avg(uint8_t ch, uint8_t n){
        uint32_t s=0; for(uint8_t i=0;i<n;i++) s+=adc_read(ch); return (uint16_t)(s/n);
    }
    static inline uint8_t pot_to_deg(uint16_t v){
        uint32_t ang = ((uint32_t)v*180)/1023; if(ang>180) ang=180; return (uint8_t)ang;
    }
    static inline uint8_t smooth_move(uint8_t a, uint8_t t){
        if(a<t){ uint8_t d=t-a; a += (d>SERVO_SLEW_STEP)?SERVO_SLEW_STEP:d; }
        else if(a>t){ uint8_t d=a-t; a -= (d>SERVO_SLEW_STEP)?SERVO_SLEW_STEP:d; }
        return a;
    }
}

// UART 115200 (U2X=1) + RX por interrupcion
static void uart_init(uint32_t baud){
    UCSRA = (1<<U2X0); // doble velocidad
    uint16_t ubrr = (uint16_t)(F_CPU/8/ baud - 1);
    UBRROH = (uint8_t)(ubrr>>8);
    UBRROL = (uint8_t)(ubrr&0xFF);
    UCSRB = (1<<RXEN0) | (1<<TXEN0) | (1<<RXCIE0); // habilita RX, TX e INT RX
    UCSRC = (1<<UCSZ01) | (1<<UCSZ00); // 8N1
}

static inline void uart_tx(uint8_t c){ while(!(UCSRA&(1<<UDRE0))); UDR0=c; }
static void uart_write(const char* s){ while(*s) uart_tx((uint8_t)*s++); }

// Ring buffer RX (tamano potencia de 2)
#define RX_BUF_SZ 128
static volatile char rx_buf[RX_BUF_SZ];
static volatile uint8_t rx_head=0, rx_tail=0;

ISR(USART_RX_vect){
    uint8_t nh = (rx_head + 1) & (RX_BUF_SZ-1);
    char c = UDR0; // leer SIEMPRE UDR0
    if(nh != rx_tail){ // si hay espacio
        rx_buf[rx_head] = c;
        rx_head = nh;
    } // si se llena, descarta byte
}

static bool rb_get_char(char *out){
    if(rx_head == rx_tail) return false;
    *out = rx_buf[rx_tail];
    rx_tail = (rx_tail + 1) & (RX_BUF_SZ-1);
    return true;
}

// Parser helpers
static inline void to_upper_str(char *s){ for(;*s;++s) if(*s>='a'&&*s<='z') *s+= 'A'- 'a'; }
static inline bool is_printable(unsigned char c){ return (c>=32 && c<=126); }
static void compact_colon_spaces(char *s){
    char t[96]; uint8_t k=0;
    for(uint8_t i=0; s[i] && k<sizeof(t)-1; ++i){
        if(s[i]==':'){
            while(k>0 && (t[k-1]==' '||t[k-1]=='\t')) k--;
            t[k++]=': '; uint8_t m=i+1; while(s[m]==' '||s[m]=='\t') m++;
            i=m-1;
        } else t[k++]=s[i];
    }
    t[k]='\0'; strcpy(s,t);
}

```

```

}

// Estado y modos
typedef enum { MODE_POT=0, MODE_REMOTE=1 } mode_t;
static volatile mode_t mode = MODE_POT;
static uint8_t angA=90, targetA=90;

// Ensamblador de lineas
static char line[96]; static uint8_t li=0;
static void process_line(char *ln){
    // limpiar no-imprimibles
    char s[96]; uint8_t j=0;
    for(uint8_t i=0; ln[i] && j<sizeof(s)-1; ++i){
        unsigned char c=(unsigned char)ln[i];
        if(is_printable(c)) s[j++]=(char)c;
    }
    s[j]='\0';
    to_upper_str(s);
    compact_colon_spaces(s);

    if (strncmp(s,"S:",2)==0){
        int v=-1;
        if (sscanf(s+2,"%d",&v)==1 && v>=0 && v<=180){
            targetA = (uint8_t)v;
            mode = MODE_REMOTE;
            angA = targetA;
            servoA_write_us(deg_to_us(angA));
            char msg[24]; snprintf(msg,sizeof(msg),"OK S: %u\n",angA);
uart_write(msg);

            } else uart_write("ERR\n");

            } else if (strncmp(s,"MODE:POT",8)==0){
                mode = MODE_POT; uart_write("OK\n");

            } else if (strncmp(s,"MODE:REMOTE",11)==0){
                mode = MODE_REMOTE; uart_write("OK\n");

            } else if (s[0]=='Q'){
                char msg[40]; snprintf(msg,sizeof(msg),"ANG=%u,MODE= %s\n",angA,(
mode==MODE_REMOTE)?"REMOTE":"POT");
                uart_write(msg);

            } else if (strncmp(s,"RAW:",4)==0){
                int us=-1;
                if (sscanf(s+4,"%d",&us)==1 && us>=SERVO_MIN_US && us<=SERVO_MAX_US
){
                    mode = MODE_REMOTE;
                    servoA_write_us((uint16_t)us);
                    uint32_t d=(uint32_t)(us - SERVO_MIN_US) * 180 / (
SERVO_MAX_US - SERVO_MIN_US);
                    if(d>180) d=180;
                    angA=(uint8_t)d; targetA=angA; uart_write("OK\n");
                } else uart_write("ERR\n");

            } else if (s[0]=='\0'){
                // linea vacia -> ignorar
            } else {
                char dbg[64]; snprintf(dbg,sizeof(dbg),"ERR RX: \"%s\"\n", s);
uart_write(dbg);
            }
        }
    }
}

```

```

int main(void){
    servo_init();
    adc_init();
    uart_init(115200);
    sei();

    servoA_write_us(deg_to_us(angA));
    _delay_ms(200);
    uart_write("READY\n");

    for(;;){
        // Consume bytes de la cola y arma lineas (CR/LF terminan linea)
        char c;
        while(rb_get_char(&c)){
            if(c=='\r' || c=='\n'){
                if(li>0){ line[li]='\0'; process_line(line); li=0; }
                } else {
                    if(li < sizeof(line)-1) line[li++]=c;
                }
            }

        // MODO POT
        if (mode == MODE_POT){
            uint16_t p = adc_read_avg(0, ADC_SAMPLES);
            uint8_t pdeg = pot_to_deg(p);
            if( (pdeg>targetA && (pdeg-targetA)>=ADC_DEADBAND) ||
                (pdeg<targetA && (targetA-pdeg)>=ADC_DEADBAND) ){
                targetA = pdeg;
            }
        }

        // Slew + PWM
        angA = smooth_move(angA, targetA);
        servoA_write_us(deg_to_us(angA));

        _delay_ms(10);
    }
}

```

B.2. Control de Dos Servomotores con Potenciómetros (Control Espejo)

```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

/* ===== CONFIGURACION SERVO ===== */
#define SERVO_MIN_US 350
#define SERVO_MAX_US 2650
#define SERVO_SLEW_STEP 2 // velocidad de respuesta
#define ADC_SAMPLES 10 // promedio de muestras para suavizar

/* ===== CONFIGURACION PWM ===== */
static inline void servo_init(void) {
    DDRB |= (1 << PB1) | (1 << PB2); // D9 y D10 como salida
    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);
    ICR1 = 39999; // 20 ms -> 50 Hz
}

```

```

static inline uint16_t us_to_ticks(uint16_t us) {
    return (uint16_t)(us * 2);
}

/* ===== FUNCIONES DE POSICION ===== */
static inline void servoA_write_us(uint16_t us) {
    if (us < SERVO_MIN_US) us = SERVO_MIN_US;
    if (us > SERVO_MAX_US) us = SERVO_MAX_US;
    OCR1A = us_to_ticks(us);
}

static inline void servoB_write_us(uint16_t us) {
    if (us < SERVO_MIN_US) us = SERVO_MIN_US;
    if (us > SERVO_MAX_US) us = SERVO_MAX_US;
    OCR1B = us_to_ticks(us);
}

/* Grados (0-180) -> microsegundos */
static inline uint16_t deg_to_us(uint8_t deg) {
    if (deg > 180) deg = 180;
    return SERVO_MIN_US + ((uint32_t)(SERVO_MAX_US - SERVO_MIN_US) * deg) / 180;
}

/* ===== ADC ===== */
static inline void adc_init(void) {
    ADMUX = (1 << REFS0); // referencia AVcc
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1); // prescaler=64
}

/* Leer canal (0-7) */
static inline uint16_t adc_read(uint8_t ch) {
    ADMUX = (ADMUX & 0xF0) | (ch & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}

/* Promediar varias lecturas */
static inline uint16_t adc_read_avg(uint8_t ch, uint8_t muestras) {
    uint32_t suma = 0;
    for (uint8_t i = 0; i < muestras; i++) {
        suma += adc_read(ch);
    }
    return (uint16_t)(suma / muestras);
}

/* ===== MAPEO POT -> ANGULO ===== */
static inline uint8_t pot_to_deg(uint16_t val) {
    uint32_t ang = ((uint32_t)val * 180) / 1023;
    if (ang > 180) ang = 180;
    return (uint8_t)ang;
}

/* ===== SLEW (movimiento suave) ===== */
static inline uint8_t smooth_move(uint8_t actual, uint8_t target) {
    if (actual < target) {
        uint8_t diff = target - actual;
        if (diff > SERVO_SLEW_STEP) actual += SERVO_SLEW_STEP;
        else actual = target;
    } else if (actual > target) {
        uint8_t diff = actual - target;

```

```

        if (diff > SERVO_SLEW_STEP) actual -= SERVO_SLEW_STEP;
        else actual = target;
    }
    return actual;
}

/* ===== MAIN ===== */
int main(void) {
    servo_init();
    adc_init();

    uint8_t angA = 90; // servo en D9
    uint8_t angB = 90; // servo en D10

    servoA_write_us(deg_to_us(angA));
    servoB_write_us(deg_to_us(angB));
    _delay_ms(500);

    while (1) {
        // Leer potenciómetros A0 y A1
        uint16_t potA = adc_read_avg(0, ADC_SAMPLES);
        uint16_t potB = adc_read_avg(1, ADC_SAMPLES);

        // Mapear a ángulos
        uint8_t targetA = pot_to_deg(potA);
        uint8_t targetB = pot_to_deg(potB);

        // Movimiento suave
        angA = smooth_move(angA, targetA);
        angB = smooth_move(angB, targetB);

        // Actualizar servos
        servoA_write_us(deg_to_us(angA));
        servoB_write_us(deg_to_us(angB));

        _delay_ms(15);
    }
}

```

B.3. Programa Principal del Brazo Robótico Didáctico

```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/eeprom.h>
#include <util/delay.h>
#include <stdlib.h>
#include <string.h>

/* ==== Pines ==== */
#define LED_ROJO    PD4
#define LED_VERDE   PD5
#define LED_AZUL    PD7
#define BUZZER      PD3
#define ELECTRO     PB0

#define SERVO_BASE_PIN    PB1    // D9  (OC1A)
#define SERVO_HOMBRO_PIN  PB2    // D10 (OC1B)
#define SERVO_CODO_PIN    PB3    // D11
#define SERVO_PINZA_PIN   PD6    // D6
#define MODE_PIN_PC4      PC4    // A4 modo físico

```



```

/* ==== Constantes servo ==== */
#define SERVO_MIN_US    500
#define SERVO_MAX_US    2500
#define SERVO_MIN_DEG   0
#define SERVO_MAX_DEG   180

#define T2_PRESC_8_TICK_US    32UL
#define SERVO_PERIOD_US      20000UL
#define T2_COUNTS_PER_PERIOD (SERVO_PERIOD_US / T2_PRESC_8_TICK_US)

/* ==== Variables globales ==== */
static uint8_t vel_ms    = 20;
static uint8_t paso_ang  = 1;

static uint8_t base_min=30, base_max=150;
static uint8_t hombro_min=40,hombro_max=140;
static uint8_t codo_min=30, codo_max=150;
static uint8_t pinza_min=20, pinza_max=160;

volatile uint8_t servoCodoTicks = (1500UL / T2_PRESC_8_TICK_US);
volatile uint8_t servoPinzaTicks = (1500UL / T2_PRESC_8_TICK_US);
volatile uint16_t sw_counter = 0;
volatile uint8_t modo_serial = 0; // 0=sin comando, 1>manual, 2=auto

/* ==== EEPROM ==== */
uint8_t EEMEM eeprom_last_mode;

/* ==== UART ==== */
#define BAUD 9600
#define MYUBRR F_CPU/16/BAUD-1

void uart_init(void) {
    uint16_t ubrr = MYUBRR;
    UBRROH = (uint8_t)(ubrr>>8);
    UBRROL = (uint8_t)ubrr;
    UCSROB = (1<<RXEN0)|(1<<TXEN0);
    UCSROC = (1<<UCSZ01)|(1<<UCSZ00);
}

void uart_tx(char data) {
    while (!(UCSROA & (1<<UDRE0)));
    UDRO = data;
}

void uart_print(const char *s) {
    while (*s) uart_tx(*s++);
}

uint8_t uart_available(void) {
    return (UCSROA & (1<<RXCO));
}

char uart_rx(void) {
    while (!(UCSROA & (1<<RXCO)));
    return UDRO;
}

/* ==== Servos ==== */
static inline uint16_t us_from_deg(uint8_t ang){
    if (ang > 180) ang = 180;
    return SERVO_MIN_US + ((uint32_t)ang * (SERVO_MAX_US - SERVO_MIN_US)) / 180;
}

static inline uint8_t ticks_from_us(uint16_t us){
    uint16_t t = us / T2_PRESC_8_TICK_US;

```

```

    if (t < 1) t = 1;
    return (uint8_t)t;
}
void servo_hw12_init(void) {
    ICR1 = 39999;
    TCCR1A = (1<<COM1A1)|(1<<COM1B1)|(1<<WGM11);
    TCCR1B = (1<<WGM13)|(1<<WGM12)|(1<<CS11);
    DDRB |= (1<<SERVO_BASE_PIN)|(1<<SERVO_HOMBRO_PIN);
}
void servo_base(uint8_t ang) { OCR1A = us_from_deg(ang) * 2; }
void servo_hombro(uint8_t ang) { OCR1B = us_from_deg(ang) * 2; }

void servo_sw_init(void) {
    DDRB |= (1<<SERVO_CODO_PIN);
    DDRD |= (1<<SERVO_PINZA_PIN);
    TCCR2A = 0;
    TCCR2B = (1<<CS21);
    TIMSK2 = (1<<TOIE2);
}
ISR(TIMER2_OVF_vect) {
    sw_counter++;
    if (sw_counter == 1) {
        PORTB |= (1<<SERVO_CODO_PIN);
        PORTD |= (1<<SERVO_PINZA_PIN);
    }
    if (sw_counter == servoCodoTicks) { PORTB &= ~(1<<SERVO_CODO_PIN); }
    if (sw_counter == servoPinzaTicks) { PORTD &= ~(1<<SERVO_PINZA_PIN); }
    if (sw_counter >= T2_COUNTS_PER_PERIOD) sw_counter = 0;
}
void servo_codo(uint8_t ang) { servoCodoTicks = ticks_from_us(us_from_deg(ang)); }
void servo_pinza(uint8_t ang) { servoPinzaTicks = ticks_from_us(us_from_deg(ang)); }

/* ==== ADC ==== */
void adc_init(void) {
    ADMUX = (1<<REFS0);
    ADCSRA = (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0);
}
uint16_t adc_read(uint8_t ch) {
    ch &= 0x07;
    ADMUX = (ADMUX & 0xF8) | ch;
    ADCSRA |= (1<<ADSC);
    while (ADCSRA & (1<<ADSC));
    return ADC;
}

/* ==== LEDs y Buzzer ==== */
static inline void leds_off(void) {
    PORTD &= ~(1<<LED_ROJO)|(1<<LED_VERDE)|(1<<LED_AZUL);
}
static inline void beep(uint8_t n) {
    for (uint8_t i=0; i<n; i++){
        PORTD |= (1<<BUZZER); _delay_ms(100);
        PORTD &= ~(1<<BUZZER); _delay_ms(100);
    }
}
void movimiento_iniciado(void){
    PORTD |= (1<<LED_VERDE);
    beep(1);
    _delay_ms(100);
    leds_off();
}

```

```

void objeto_detectado(void){
    PORTD |= (1<<LED_AZUL);
    beep(2);
    _delay_ms(100);
    leds_off();
}

void error_movimiento(void){
    PORTD |= (1<<LED_ROJO);
    _delay_ms(1000);
    leds_off();
    beep(2);
}

/* ==== Modo fisico ==== */
void mode_init(void){
    DDRC  &= ~(1<<MODE_PIN_PC4);
    PORTC |= (1<<MODE_PIN_PC4);
}

uint8_t is_manual_mode(void){
    return ( (PINC & (1<<MODE_PIN_PC4)) == 0 );
}

/* ==== Serial control ==== */
void check_serial_mode(void){
    if (uart_available()){
        char c = uart_rx();
        if (c == 'm' || c == 'M'){
            modo_serial = 1;
            eeprom_update_byte(&eeprom_last_mode, 1);
            uart_print("Modo Manual Activado\r\n");
        } else if (c == 'a' || c == 'A'){
            modo_serial = 2;
            eeprom_update_byte(&eeprom_last_mode, 2);
            uart_print("Modo Automatico Activado\r\n");
        }
    }
}

/* ==== Movimiento automatico predefinido ==== */
void movimiento_automatico(void){
    movimiento_iniciado();

    servo_base(60);    _delay_ms(600);
    servo_hombro(120); _delay_ms(600);
    servo_codo(90);    _delay_ms(600);
    servo_pinza(30);   _delay_ms(600);

    objeto_detectado();

    servo_pinza(160); _delay_ms(600);
    servo_base(90);   _delay_ms(600);
    servo_hombro(90); _delay_ms(600);
    servo_codo(90);   _delay_ms(600);
}

/* ==== MAIN ==== */
int main(void) {
    DDRD |= (1<<LED_ROJO) | (1<<LED_VERDE) | (1<<LED_AZUL) | (1<<BUZZER);
    DDRB |= (1<<ELECTRO);

    PORTD &= ~(1<<LED_ROJO) | (1<<LED_VERDE) | (1<<LED_AZUL) | (1<<BUZZER));
}

```

```

PORTB &= ~(1<<ELECTRO);

servo_hw12_init();
servo_sw_init();
adc_init();
mode_init();
uart_init();
sei();

uart_print("Sistema iniciado. Ingrese 'm' o 'a' para cambiar modo.\r\n");

uint8_t last_mode = eeprom_read_byte(&eeprom_last_mode);
if (last_mode == 1) modo_serial = 1;
else if (last_mode == 2) modo_serial = 2;

servo_base(90);
servo_hombro(90);
servo_codo(90);
servo_pinza(90);

uint8_t manual_prev = 255;

while (1){
    check_serial_mode();

    uint8_t manual;
    if (modo_serial == 1) manual = 1;
    else if (modo_serial == 2) manual = 0;
    else manual = is_manual_mode();

    if (manual != manual_prev){
        leds_off();
        if (manual){
            PORTD |= (1<<LED_VERDE); beep(1);
        } else {
            PORTD |= (1<<LED_AZUL); beep(2);
        }
        _delay_ms(150);
        leds_off();
        manual_prev = manual;
    }

    if (manual){
        uint16_t p1 = adc_read(0);
        static uint16_t p1f = 0;
        p1f = (p1f*3 + p1)/4;
        uint8_t ang = (p1f * 180UL) / 1023;
        servo_base(ang);
        _delay_ms(15);
    } else {
        movimiento_automatico();
        _delay_ms(2000);
    }
}
}

```

B.4. Interfaz de Usuario

```

import tkinter as tk
from tkinter import ttk, messagebox
import serial, time

```

```

from serial.tools import list_ports

# ===== Config =====
BAUD = 115200
SEND_INTERVAL_MS = 40      # frecuencia de envio durante arrastre
POLL_INTERVAL_MS = 150     # frecuencia de consulta Q?

# ===== Estado global =====
ser = None
connected = False
angle_var = None
last_sent_deg = {"v": None}
pending_deg = {"v": 90}
send_after_id = {"id": None}
poll_after_id = {"id": None}

# ===== Serial helpers =====
def write_line(line: str):
    global ser, connected
    if not connected or ser is None:
        return
    try:
        ser.write((line + "\n").encode("ascii", errors="ignore"))
        ser.flush()
    except Exception as e:
        last_msg_var.set(f"TX error: {e}")

def read_available():
    global ser, connected
    if not connected or ser is None:
        return ""
    out = b""
    try:
        chunk = ser.read(ser.in_waiting or 1)
        out += chunk
        time.sleep(0.002)
        while ser.in_waiting:
            out += ser.read(ser.in_waiting)
    except Exception:
        pass
    return out.decode(errors="ignore")

def parse_and_update(lines: str):
    """Procesa lineas y actualiza S1, Modo y el label de ultimo mensaje."""
    last_line = None
    for line in lines.splitlines():
        line = line.strip()
        if not line:
            continue
        last_line = line
        if line.startswith("ANG="):
            try:
                parts = line.split(",", 1)
                ang_part = parts[0].split("=")[1]
                mode_part = parts[1].split("=")[1] if len(parts) > 1 else ""
                servo_pos_var.set(f"S1: {ang_part} grados")
                mode_var.set(f"Modo: {mode_part}")
            except Exception:
                pass
    if last_line:
        last_msg_var.set(f"Ultimo: {last_line}")

```

```

# ===== UI callbacks =====
def refresh_ports():
    ports = [p.device for p in list_ports.comports()]
    port_combo["values"] = ports
    if ports and port_combo.get() == "":
        port_combo.set(ports[0])

def connect_serial():
    global ser, connected
    if connected:
        disconnect_serial()
        return
    port = port_combo.get().strip()
    if not port:
        messagebox.showwarning("Puerto", "Selecciona un puerto primero.")
        return
    try:
        ser = serial.Serial(port, BAUD, timeout=0.2)
        time.sleep(0.4) # espera al reset del UNO
        connected = True
        connect_btn.config(text="Desconectar")
        status_var.set(f"Conectado a {port} @ {BAUD} baud")
        pending = read_available().strip()
        if pending:
            parse_and_update(pending)
        start_polling()
    except Exception as e:
        ser = None
        connected = False
        messagebox.showerror("Error", f"No se pudo abrir {port}\n{e}")

def disconnect_serial():
    global ser, connected
    stop_polling()
    if ser:
        try:
            ser.close()
        except Exception:
            pass
    ser = None
    connected = False
    connect_btn.config(text="Conectar")
    status_var.set("Desconectado")
    last_msg_var.set("Ultimo: -")

def start_polling():
    def _tick():
        if not connected:
            return
        write_line("Q?")
        root.after(20, _read_after_q)
        poll_after_id["id"] = root.after(POLL_INTERVAL_MS, _tick)
    _tick()

def _read_after_q():
    resp = read_available().strip()
    if resp:
        parse_and_update(resp)

def stop_polling():

```

```

    if poll_after_id["id"] is not None:
        root.after_cancel(poll_after_id["id"])
        poll_after_id["id"] = None

# ===== Envio continuo mientras se mueve el slider =====
def _send_pending():
    send_after_id["id"] = None
    deg = int(pending_deg["v"])
    if deg != last_sent_deg["v"]:
        write_line(f"S:{deg}")
        last_sent_deg["v"] = deg
        last_msg_var.set(f"Ultimo: > S:{deg}")
    if connected:
        send_after_id["id"] = root.after(SEND_INTERVAL_MS, _send_pending)

def on_slider_change(_val_str=None):
    pending_deg["v"] = angle_var.get()
    if connected and send_after_id["id"] is None:
        send_after_id["id"] = root.after(SEND_INTERVAL_MS, _send_pending)

def on_release(_event=None):
    if send_after_id["id"] is not None:
        root.after_cancel(send_after_id["id"])
        send_after_id["id"] = None
    deg = angle_var.get()
    if deg != last_sent_deg["v"]:
        write_line(f"S:{deg}")
        last_sent_deg["v"] = deg
        last_msg_var.set(f"Ultimo: > S:{deg}")

def set_mode_pot():
    write_line("MODE:POT")

# ===== UI =====
root = tk.Tk()
root.title("Control de Servomotor (Arduino UNO)")

main = ttk.Frame(root, padding=12)
main.pack(fill="both", expand=True)
root.rowconfigure(0, weight=1)
root.columnconfigure(0, weight=1)

# Fila 0: Puerto serie
ttk.Label(main, text="Puerto:").grid(row=0, column=0, sticky="w")
port_combo = ttk.Combobox(main, state="readonly", width=18)
port_combo.grid(row=0, column=1, sticky="w")
ttk.Button(main, text="Refrescar", command=refresh_ports).grid(row=0, column=2, sticky="w",
    padx=6)
connect_btn = ttk.Button(main, text="Conectar", command=connect_serial)
connect_btn.grid(row=0, column=3, sticky="e")

# Fila 1: Estado conexion + Ultimo mensaje
status_var = tk.StringVar(value="Desconectado")
ttk.Label(main, textvariable=status_var).grid(row=1, column=0, columnspan=2, sticky="w",
    pady=(6,0))
last_msg_var = tk.StringVar(value="Ultimo: -")
ttk.Label(main, textvariable=last_msg_var).grid(row=1, column=2, columnspan=2, sticky="e",
    pady=(6,0))

# Fila 2: Bloques de estado (S1 + Modo)
servo_pos_var = tk.StringVar(value="S1: -grados")

```

```

mode_var = tk.StringVar(value="Modo: -")
servo_pos_lbl = ttk.Label(main, textvariable=servo_pos_var, font=("Segoe UI", 12, "bold"))
servo_pos_lbl.grid(row=2, column=0, sticky="w", pady=(10,0))
mode_lbl = ttk.Label(main, textvariable=mode_var, font=("Segoe UI", 12, "bold"))
mode_lbl.grid(row=2, column=1, sticky="w", pady=(10,0))

# Fila 3: Slider y boton de modo
angle_var = tk.IntVar(value=90)
ttk.Label(main, text="Angulo (grados)").grid(row=3, column=0, sticky="w", pady=(12,0))
s = ttk.Scale(main, from_=0, to=180, orient="horizontal",
               variable=angle_var, length=300, command=on_slider_change)
s.grid(row=3, column=1, sticky="ew", padx=8, pady=(12,0))
s.bind("<ButtonRelease-1>", on_release)

ttk.Button(main, text="Modo POT", command=set_mode_pot).grid(row=3, column=2, sticky="w",
                    padx=6, pady=(12,0))

# Inicializar lista de puertos
refresh_ports()

root.mainloop()

# Cierre limpio
try:
    disconnect_serial()
except Exception:
    pass

```